SNAIL CLUB / Proceso y decisiones

Borrador de entrega. Pendiente: URL pública verificada de GitHub. El despliegue público se atenderá posteriormente.

Proceso

Se desglosaron los requisitos en acceso local, dashboard, pasarela simulada, manejo de errores y entregables. Se implementó primero el registro y la sesión, después el saldo y los contratos de SnailPay, y finalmente la interfaz, las pruebas automatizadas y la documentación. Se revisaron capturas de escritorio y móvil y se ajustaron contraste y separación de etiquetas. La compilación se verificó por separado de los recorridos del navegador.

Decisiones principales

React y TypeScript para el frontend; Express y TypeScript para el backend. LocalStorage conserva la cuenta, sesión, saldo y operaciones. Los componentes acceden a servicios, y la persistencia queda aislada mediante un repositorio. SnailPay es un mock, sin integración financiera externa. El dashboard usa seis carreras fijas de un único día simulado para derivar victorias y apuestas coherentes. No hay calendario, ejecución de carreras ni movimientos de saldo asociados a esas estadísticas.

La contraseña no se guarda en texto plano: se deriva con PBKDF2-SHA256, una sal aleatoria y 210 000 iteraciones. Esta decisión reduce la exposición directa, pero no convierte el login local en autenticación de servidor. Los importes se calculan en centavos; saldo y operación se guardan en una misma escritura. El cliente solo acredita respuestas aprobadas, coherentes con la solicitud y guardadas correctamente.

Los datos de tarjeta y CVV son fixtures conocidas y se incluyen en respuestas y almacenamiento local de esta simulación. No se aceptan números o CVV ajenos a la lista permitida. El caso aprobado conserva la fecha fija 12/26, sin validación bancaria real. Estas excepciones no se trasladan a una futura integración financiera.

Herramientas, diseño e IA

Vite, React Router, Recharts, Zod, Phosphor Icons y DM Sans. Diseño implementado en CSS a partir de una referencia visual proporcionada por el usuario. Se adaptaron el menú, barra superior, panel central y colores a saldo y resultados simulados. La interfaz ocupa toda la ventana; marca y cierre de sesión permanecen fijos y el contenido se desplaza por separado. El menú contiene Dashboard y Cargar saldo. Ambas gráficas están en el dashboard. Su botón de recarga abre un modal con foco contenido, cierre con Escape y devolución del foco. El mismo formulario aparece en Cargar saldo, con el historial a su derecha en escritorio y debajo en móvil. No se utilizó una plantilla descargada. Recharts resuelve las gráficas. La tipografía se sirve localmente. Vitest y Supertest verifican lógica y API; Playwright recorre la aplicación; Prettier uniforma el formato.

Se utilizó inteligencia artificial de principio a fin mediante Codex para el análisis técnico, la arquitectura, el diseño, la generación y corrección del código, las pruebas y la documentación. Leí personalmente el documento de requisitos y fui guiando a la IA durante el desarrollo de la aplicación: definí el alcance, aporté la referencia visual, revisé los resultados y solicité correcciones de navegación, distribución e interacción. La implementación del código, las pruebas y los documentos se realizó con IA; no se presenta como escritura manual independiente.

Se utilizaron la guía emil-design-eng para accesibilidad y detalles de interacción, y la habilidad PDF para generar y revisar el documento. El proceso incluyó requisitos explícitos, separación de responsabilidades, contratos tipados, validaciones, pruebas automatizadas, revisión visual e instalación desde una copia limpia. Estas comprobaciones aportan evidencia del funcionamiento; no garantizan por sí solas la ausencia de errores. No se utilizó Superpowers ni subagentes. La revisión y comprensión personal del código siguen siendo responsabilidad del autor.

El PDF se genera con Python y ReportLab, en Arial de 10 puntos. No incluye código ni capturas.

SNAIL CLUB / Validación y alcance

Pruebas y razón de su elección

La lógica de registro, verificación de contraseña, logout/login, saldo inicial, sumas de centavos, persistencia, rechazo y deduplicación se prueba porque define el estado de la cuenta. El almacenamiento dañado y la escritura fallida se prueban para evitar pérdidas silenciosas o confirmaciones falsas. La API se prueba con aprobación, rechazo, validación de montos, JSON malformado, referencias repetidas o en conflicto y fallos de servicio.

El transporte se prueba con respuestas incoherentes, datos inválidos, desconexión y cancelación por timeout. Los recorridos completos verifican registro, acceso protegido, recarga, recarga de página, historial, cierre y reapertura de sesión. También verifican que ninguna variante de error acredite saldo, la sincronización entre pestañas, las vistas separadas, el scroll independiente y el teclado del modal. Los mismos escenarios se ejecutan en escritorio y en un viewport móvil de Chromium; esto no acredita compatibilidad con Safari o con un teléfono físico.

Funcionalidades terminadas

Registro validado; inicio/cierre de sesión; persistencia local; protección de rutas; perfil y saldo; dos vistas: dashboard con donut de apuestas y barras de victorias, y carga de saldo con historial; seis caracoles y seis carreras coherentes; recarga aprobada; rechazo; fallo interno; timeout; mensajes de error; historial de respuestas; tarjeta/CVV ficticios persistidos; prevención local de duplicados; pruebas y documentación de ejecución. Se elaboró la propuesta adicional de PostgreSQL sin implementar una base de datos.

Pendientes y limitaciones

Pendientes de cierre: autor del commit, publicación del repositorio, URL verificada de GitHub. No hay despliegue público; es un adicional no implementado. PostgreSQL está propuesto, no conectado. No hay gestión multiusuario, recuperación de contraseña, verificación de correo ni apuestas/carreras ejecutables.

LocalStorage puede ser modificado por quien controla el navegador. No existe autenticación de servidor ni saldo financiero confiable. Borrar los datos del sitio elimina la cuenta. El caché de SnailPay es temporal, limitado a 1 000 referencias y se pierde al reiniciar. No hay conciliación durable tras un cierre inesperado. Web Locks coordina pestañas cuando está disponible; la alternativa solo serializa una instancia. En esta demo, un timeout simulado nunca aprueba; un proveedor real requeriría consulta y conciliación del estado desconocido.

Ejecución y entrega

Node.js 22.12 o posterior. Instalar dependencias con npm ci y ejecutar npm run dev. Interfaz en 127.0.0.1:5173; API en 127.0.0.1:3001. npm run build compila ambos proyectos; npm start sirve la compilación desde el puerto 3001. npm test ejecuta pruebas de lógica/API; instalar Chromium de Playwright y ejecutar npm run test:e2e para los recorridos completos. README y docs/SNAILPAY.md contienen los comandos y escenarios exactos, incluidos cinco comandos de demostración que imprimen el HTTP y la respuesta de SnailPay. docs/GUIA_CODIGO.md explica el recorrido de registro, sesión y recarga, los límites y la razón de las pruebas.

Repositorio público: pendiente de publicación y verificación. Nombre preparado: alberto-4181, con cuatro cifras generadas aleatoriamente; su disponibilidad aún no se ha comprobado. El proyecto usa la rama main. Se revisaron los archivos publicables para excluir el documento fuente, datos locales, credenciales y referencias identificadoras del origen de los requisitos.

Tiempo aproximado invertido: 5 horas, según la estimación del responsable. Incluye la lectura de requisitos, la dirección del desarrollo con IA, la revisión de resultados y los ajustes de la aplicación y sus entregables.

Resultados de comprobación local

Compilación de frontend y backend aprobada. 30 pruebas de lógica e integración aprobadas; 10 recorridos completos aprobados, repartidos entre escritorio y móvil. Se verificaron el servidor compilado y sus rutas, una instalación limpia y cinco escenarios por API. La revisión visual cubrió acceso, dashboard y recarga. Estos resultados no prueban despliegue, funcionamiento en Safari ni persistencia SQL.

Preparación antes de publicar

Revisar personalmente el código y sus decisiones; confirmar la identidad del autor del commit; crear el repositorio público con el nombre preparado; publicar solo los archivos del proyecto; verificar su acceso público; sustituir el estado pendiente por la URL real y regenerar este documento. No se incluye el documento de requisitos en el repositorio.

Adicional: propuesta de PostgreSQL

PostgreSQL almacenaría usuarios, sesiones, wallets y operaciones. Un usuario tendría varias sesiones y una wallet por moneda; una wallet tendría múltiples operaciones. Correo normalizado único, claves foráneas, referencias únicas, importes enteros de centavos y fechas UTC. No se guardarían CVV ni números completos de tarjeta.

Express asumiría la autoridad sobre credenciales, sesiones y saldo. El frontend enviaría solicitudes de negocio y consultaría el saldo del servidor, sin enviar un valor local para que fuera aceptado como verdadero. Los servicios de interfaz pasarían a usar HTTP y los repositorios SQL existirían solo en el backend. Credenciales de PostgreSQL solo en variables del servidor. Las sesiones se manejarían con cookies protegidas, expiración y revocación; las contraseñas se derivarían en servidor.

Una recarga se registraría como pendiente con referencia única antes de consultar al proveedor. Confirmado el resultado, una transacción SQL corta bloquearía la operación y wallet, comprobaría que no se aplicó antes y guardaría el movimiento junto con el incremento del saldo. Los timeouts reales quedarían pendientes de conciliación, no se tratarían como rechazos definitivos. La unicidad persistente impediría dobles acreditaciones incluso tras reiniciar.

Las carreras seguirían como fixtures hasta necesitar historial. La integración futura requiere migraciones, repositorios, endpoints autenticados y pruebas de concurrencia, rollback y aislamiento entre usuarios. No basta con cambiar un adaptador para obtener estas garantías. El diseño ampliado está en docs/POSTGRESQL.md.